

ECON*6140 ECONOMETRICS I

Python Cheat Sheet for HW1

Matrices only — no regression packages

How to use it. Copy one grey block into one notebook cell, change the names, press **Shift+Enter**. One idea per cell, so when something breaks you know where. Run cells top to bottom.

1 Five rules

1. @ is matrix multiplication. * multiplies element by element. Mixing them up gives a wrong answer with *no* error.
2. X.T is X' . np.linalg.inv(A) is A^{-1} . np.eye(n) is the $n \times n$ identity I .
3. Python counts from 0. b[0] is the first coefficient (the constant), b[1] the second.
4. df['r'] picks a column (square brackets). df.head() runs an action (round brackets).
5. X.shape tells you the size of a matrix. Print it whenever you are unsure.

2 Load the data and make variables

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.set_printoptions(suppress=True)      # show 0.0161, not 1.61e-02

df = pd.read_excel('tbrate.xlsx', sheet_name='data')
df.index = pd.PeriodIndex(df['period'], freq='Q')  # rows named 1950Q1, 1950Q2, ...
```

Two tools make every variable in the assignment:

Code	Means	Example
.diff()	change from last period, $\Delta x_t = x_t - x_{t-1}$	df['r'].diff()
.shift(1)	last period's value, x_{t-1}	df['pi'].shift(1)
.shift(2)	two periods back, x_{t-2}	df['dr'].shift(2)
np.log()	natural log	np.log(df['c'])

```
df['dr']   = df['r'].diff()      # change in r
df['dy']   = df['y'].diff()      # change in y
df['pi_1'] = df['pi'].shift(1)   # last quarter's pi
df['dy_1'] = df['dy'].shift(1)   # last quarter's dy (needs dy made first!)
df['dr_1'] = df['dr'].shift(1)   # last quarter's dr
df['dr_2'] = df['dr'].shift(2)   # dr two quarters back
```

Lags leave empty cells (NaN) at the top. That is why the sample starts later. Cut the sample by date (both ends included):

```
sub = df.loc['1950Q4':'1996Q4']
len(sub)      # 185 rows
sub.isna().sum()  # every column should say 0
```

3 Regression with matrices

Step 1: turn columns into y and X .

The constant is just a column of ones.

```
one = np.ones(len(sub))           # the constant
y = sub['dr'].to_numpy()           # y as a plain vector
X = np.column_stack([one, sub['pi_1'], sub['dy_1'], sub['dr_1'], sub['dr_2']])
X.shape                           # (185, 5)
```

`np.column_stack(...)` glues columns side by side. The order you list them is the order of b .

Step 2: the formulas.

Math	Code	Name
$b = (X'X)^{-1}X'y$	<code>b = np.linalg.inv(X.T @ X) @ X.T @ y</code>	coefficients
$\hat{y} = Xb$	<code>yhat = X @ b</code>	fitted values
$e = y - \hat{y}$	<code>e = y - yhat</code>	residuals
$s^2 = e'e / (n - k)$	<code>s2 = (e @ e) / (n - k)</code>	error variance
$\widehat{\text{Var}}(b) = s^2(X'X)^{-1}$	<code>V = s2 * np.linalg.inv(X.T @ X)</code>	covariance matrix
$\text{se}(b_j) = \sqrt{V_{jj}}$	<code>se = np.sqrt(np.diag(V))</code>	standard errors

Step 3: write it once, reuse it.

Put this in one cell and run it. After that, every regression in the assignment is one line.

```
def ols(y, X):
    XtX_inv = np.linalg.inv(X.T @ X)
    b = XtX_inv @ X.T @ y           # coefficients
    yhat = X @ b                    # fitted values
    e = y - yhat                    # residuals
    n, k = X.shape                  # rows, columns
    s2 = (e @ e) / (n - k)
    V = s2 * XtX_inv                # Var(b)
    se = np.sqrt(np.diag(V))        # standard errors
    return b, se, yhat, e, V
```

```
b, se, yhat, e, V = ols(y, X)
print(b)                          # [const, pi_1, dy_1, dr_1, dr_2]
print(se)
```

The t-statistic is `b / se`. You can leave pieces you don't need with `_`, e.g. `_, _, _, e2, _ = ols(y2, X2)` keeps only the residuals.

Regressing one vector on another.

Anything you computed (residuals, fitted values) is just a vector, so it goes straight into `column_stack`:

```
Z = np.column_stack([one, yhat])    # a constant and the fitted values
b_z, se_z, _, _, _ = ols(e, Z)      # regress e on them
```

No constant? Leave out one. With a single regressor, make it a one-column matrix: $X = v.reshape(-1, 1)$.

4 Plot against time

```
t = sub.index.to_timestamp()          # dates for the x-axis

fig, ax = plt.subplots(2, 1, figsize=(9, 6), sharex=True)
ax[0].plot(t, yhat); ax[0].set_ylabel('fitted')
ax[1].plot(t, e); ax[1].set_ylabel('residual')
ax[1].axhline(0, color='grey')        # zero line
plt.show()
```

Save it for your write-up with `fig.savefig('q2.png', dpi=200)`.

5 Projection matrices P and M

```
X1 = np.column_stack([one, sub['dy_1']])    # any group of columns
P1 = X1 @ np.linalg.inv(X1.T @ X1) @ X1.T    #  $P1 = X1 (X1'X1)^{-1} X1'$ 
M1 = np.eye(len(sub)) - P1                  #  $M1 = I - P1$ 
```

These are $n \times n$ (185×185). Use them just like the math: $P1 @ y$, $M1 @ y$, $M1 @ X2$. Handy facts to check your code: $P1y$ is the fitted value and $M1y$ the residual from regressing y on X_1 .

6 A combination of coefficients: $\hat{\gamma} = w'b$

Make w the same length as b , with a 1 on each coefficient you want to add up.

```
w = np.zeros(len(b))    # [0, 0, 0, 0, 0]
w[1] = 1                 # pick b[1]
w[2] = 1                 # pick b[2]      -> w = [0, 1, 1, 0, 0]
                        # (w[1:4] = 1 would pick b[1], b[2], b[3])

gamma = w @ b            # w'b
se_gamma = np.sqrt(w @ V @ w) # sqrt(w' Var(b) w)
```

Reparameterising.

To get the same number straight out of a regression, build new columns (e.g. `sub['x'] - sub['z']`), put them in a new X , and run `ols` again. The fitted values must not change — use that as your check.

7 Checking your answers

```

np.allclose(a, c)           # True if two vectors/matrices are equal (up to rounding)
np.allclose(P1 @ P1, P1)    # a projection times itself is itself -> True
np.allclose(M1 @ X1, 0)     # M1 wipes out X1 -> True
e.mean()                   # about 0 when X has a constant

```

Use `np.allclose`, not `==`: computers round, so two “equal” numbers can differ in the 15th decimal.

8 Errors you will hit

Message	Fix
<code>KeyError: 'dr'</code>	That column doesn't exist yet. Run the cells from the top (Kernel → Restart & Run All).
<code>matmul: ... mismatch</code>	Sizes don't fit. Print <code>X.shape</code> and <code>y.shape</code> .
<code>Singular matrix</code>	Two columns are the same thing (e.g. one added twice).
<code>nan in your answer</code>	A NaN slipped into the sample. Check <code>sub.isna().sum()</code> .
<code>NameError: name 'np'</code>	Run the import cell first.
<code>Nothing is shown</code>	Only the <i>last</i> line of a cell is displayed. Use <code>print(...)</code> .